POKÉMON TEAM BUILDER

I. DESIGN FIRST E CONNEXION

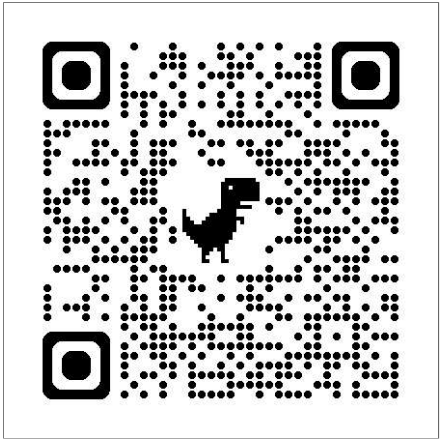
- Adotta la metodologia **Design First**:
  - definiamo prima il contratto API in un file YAML (OpenAPI Specification) e poi implementiamo il backend Python per rispettare tale contratto.
- Usiamo **Connexion (Python)** come il framework che funge da ponte:
  - Legge il file YAML.
  - Gestisce il routing automatico (collegando percorsi e metodi alle funzioni Python).
  - Esegue la **validazione automatica** del corpo JSON della richiesta contro lo schema definito nello YAML.

Come possiamo testare qualcosa che non abbiamo ancora costruito?

Mappa il codice YAML ad una tipizzazione di dati in Python



2. SPECIFICA DELL'ENDPOINT: POST /API/TEAMS



Questo endpoint è il punto di partenza per l'esercitazione e stabilisce le regole di base. https://gist.github.com/hide-ous/1341c9a8bfe5712e6dd3434e5f31db3

- [ ]

2. Specifica dell'endpoint: `POST /api/teams`

Questo endpoint è il punto di partenza per l'esercitazione e stabilisce le regole di base.

- **Metodo:** `POST`
- **Percorso:** `/api/teams`
- **Scopo:** Creare una nuova risorsa Team.
- **Body Obbligatorio (Input):** JSON contenente `name` (stringa), `trainer` (stringa), e `pokemon_names` (array di stringhe).
- **Risposta di Successo:** `201 Created` con il body della risposta che restituisce il nuovo team con l'ID univoco assegnatogli.

3. IMPLEMENTAZIONE BACKEND PYTHON (`TEAM_OPERATIONS.PY`)

- Salva la logica nel file `team_operations.py`
- Specifica l'implementazione tramite il campo **`operationId`** nel file YAML (`team_operations.create_team`).

```python
TEAMS_DATABASE = {}

# La funzione riceve il body JSON già validato da Connexion
def create_team(body):
    # La logica si concentra solo sul salvataggio dei dati
    team_data = body

    # Simulazione di generazione ID e salvataggio nel "database" (TEAMS)
    team_id = str(uuid.uuid4())
    team_data['id'] = team_id
    TEAMS[team_id] = team_data

    # Restituisce il body e il codice HTTP (201)
    return team_data, 201
```

uuid è un modulo Python

Non dovrebbe essere TEAMS_DATABASE[team_id)

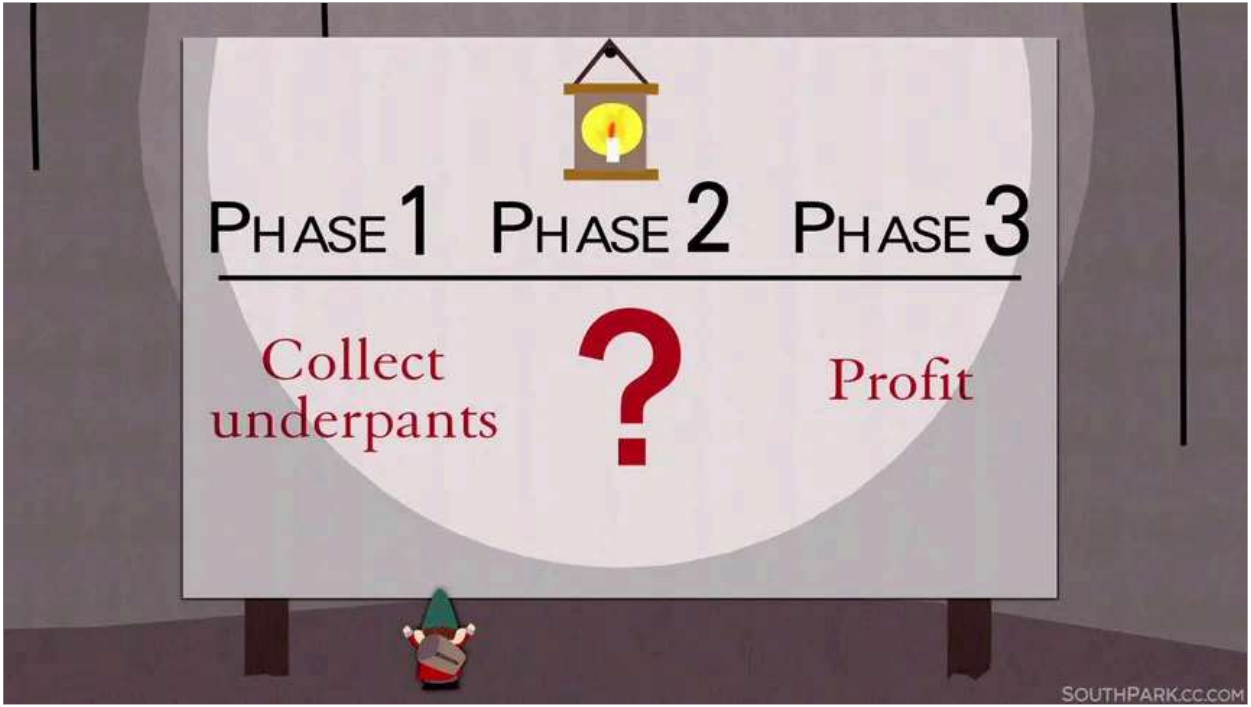
PROFIT: CONNEXION

La funzione Python non deve preoccuparsi della validazione del body JSON: Connexion garantisce che i dati siano conformi allo schema prima di invocare la logica di business.

```
import connexion
app = connexion.App(__name__, specification_dir='.')

app.add_api(
    'poketeam.yaml',
    base_path='/api',
    strict_validation=True,
    validate_responses=True
)

if __name__ == '__main__':
    app.run(port=8080)
```

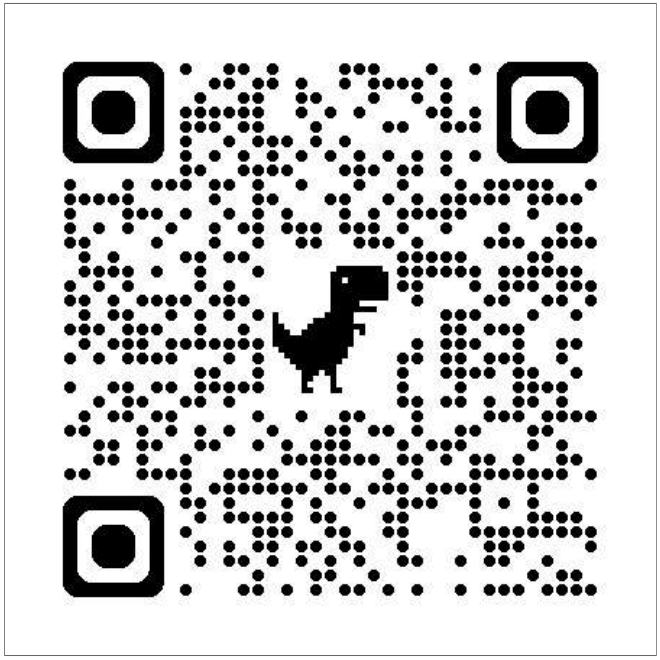

PHASE 1
PHASE 2
PHASE 3
Collect underpants
?
Profit
SOUTHPARK.cc.com

TEST

- La documentazione API è disponibile all'indirizzo http://127.0.0.1:8080/api/ui
- Testa con cURL:

```
curl -X POST http://127.0.0.1:8080/api/teams \
-H "Content-Type: application/json" \
-d '{
        "name": "Team Blaze",
        "trainer": "Giovanni",
        "pokemon_names": [  "Charizard",
                            "Arcanine",
                            "Ninetales"]
     }'
```





- [ ] https://github.com/hide-ous/WASA/tree/master/server_design_first

ESERCITAZIONE

ATTIVITÀ 1: ENDPOINT DETTAGLIO TEAM (LETTURA)

Obiettivo: Recuperare i dati di un Team specifico.

1. **YAML:** Definire il nuovo `path` `GET /teams/{team_id}` e specificare il parametro di path (`team_id`) come richiesto.
2. **YAML:** Definire le risposte appropriate (`200 OK` e `404 Not Found`).
3. **Python:** Implementare la funzione `get_team_by_id(team_id)` che recupera il team dall'archivio (l'oggetto `TEAMS`) o restituisce un errore HTTP 404 se non trovato.

- [ ]

ATTIVITÀ 2: COMPONENTE RIUSABILE PER POKÉMON

Obiettivo: Introdurre il concetto di riuso degli schemi (Components) e migliorare la struttura del body `POST`.

1. **YAML:** Creare un nuovo schema in `components/schemas` chiamato **`PokemonInstance`** con proprietà strutturate (es. `name` e `nickname`).
2. **YAML:** Modificare lo schema di richiesta `POST /teams` per sostituire l'array di stringhe con un array di `$ref: '#/components/schemas/PokemonInstance'`.
3. **Python:** Aggiornare la funzione `create_team` per gestire i nuovi dati strutturati.

- [ ]

ATTIVITÀ 3: PIGGYBACKING SU POKÉAPI (INTEGRAZIONE ESTERNA)

Obiettivo: Creare un endpoint che chiama un'API esterna (`https://pokeapi.co`) per arricchire i dati restituiti.

1. **YAML:** Definire l'endpoint annidato: `GET /teams/{team_id}/pokemons/{pokemon_name}`.
2. **Python:** Implementare la funzione `get_pokemon_details(team_id, pokemon_name)`.
3. **Logica Piggybacking:** All'interno di questa funzione, usare la libreria **`requests`** per:
    - Verificare l'esistenza del Team interno.
    - Effettuare una chiamata `GET` alla PokéAPI (es. `/api/v2/pokemon/{pokemon_name}`).

▢

- Combinare i dati ottenuti da PokéAPI (es. altezza, peso) con i dati interni del Team.
- Restituire un JSON che contenga entrambe le informazioni.

- [ ]